WinDrift++: A Diversified Detector for Concept Drift

Naureen Naqvi, *Member, IEEE*, Sabih ur Rehman, *Member, IEEE*, and Md Zahidul Islam

***Abstract*—With the rise of hyperconnectivity, vast amounts of time series data are generated by applications, many of which are business-critical and are used by incremental learning models to make informed decisions. A key requirement for such models is the ability to detect concept drift changes in the statistical characteristics of data over time. A recent method, WinDrift (WD), effectively detects drift when only a single statistical characteristic changes. However, real-world datasets often exhibit changes across multiple characteristics simultaneously, requiring a more robust detection approach. We propose WinDrift++ (WD++), a novel method that extends WD by employing a committee of statistical tests rather than a single test. This enhances robustness and accuracy in detecting diverse types of drift. WD++ retains the efficient windowing strategy of WD while improving reliability across a broader range of scenarios. We evaluate WD++ on 22 synthetic and eight real-world datasets, where it consistently outperforms WD and other state-of-the-art detectors.**

***Impact Statement*—Businesses in today's dynamic environment are seeking methods to respond to constantly evolving trends in an agile and timely manner. A major challenge faced by these businesses is the ability to detect change in emerging concepts, enabling them to use strategies for incremental learning. Concept drift detection is a well-known area of consideration in the design of incremental learning models. A drift detector is generally perceived as an essential component of a learning model. We present an advanced drift detection method *WD++* utilizing a committee of multiple statistical tests rather than a single test. The proposed method enhances the reliability, robustness, and efficiency of drift detection. *WD++* has been rigorously evaluated on 22 synthetic and eight real datasets. It demonstrates superior performance in terms of drift detection accuracy and resource utilization. The code and synthetic datasets used in this work will be made available via GitHub link https://github.com/naureenaqvi/windriftplusplus for the benefit of the research and business community. Our drift detector is suitable for deployment with minimal modifications (with appropriate context and attention) to support real-world incremental learning models across various domains of knowledge.**

***Index Terms*—Concept drift detection, data streams, ensemble methods, incremental learning, nonstationary environments, statistical hypothesis testing.**

I. Introduction

TODAY's hyperconnected world consists of large-scale applications that produce a high volume of time series data collected over time [1]. This data help real-world applications to learn incrementally over time [2]. However, this learning may be impacted by a change in time series that may occur over time, called *concept drift*. A drift may emerge at an unknown rate, resulting in inaccurate and delayed detection. These challenges may also adversely impact timely and accurate decision-making from a learning algorithm and further reinforce the need to use a concept drift detector [3]. To understand the significance of a concept drift detector, it is important to understand the impact of concept drift on learning algorithms. Generally, the nonincremental learning algorithms that have been previously trained on an old concept may face a likely negative impact on their performance with the emergence of a concept drift [4]. On the other hand, with concept drift detection, the models built from incremental learning algorithms continue to perform by adapting to the changing concepts, as shown in Fig. 1.

We refer to the literature to explain the detection of concept drift that has been previously approached in several ways. To draw insights into the characteristics of an ideal detector, we analyzed the existing techniques into three categories based on their limitations: it was noted that the first group of detectors generally had a higher rate of false alarms than the other detectors [4], [5], [6], [7], [8], whereas the second group of drift detection techniques detected drift at the expense of high computational cost [9], [10], [11], [12], and the last group of techniques heavily relied on the predefined thresholds and parameters that needed to be initialized before detecting drift, which is often not possible for data arriving from real-world applications [13], [14], [15], [16], [17], [18]. In addition to the above challenges, we analyzed the literature on the use of concept drift detectors for the detection of time series data. Recent studies [19], [20] confirmed that the state-of-the-art drift detectors presented significant limitations in detecting drift in time series data. Through this analysis, we conclude that an ideal detector must be reliable, robust, and efficient to detect

Received 28 October 2024; revised 28 June 2025 and 12 August 2025; accepted 17 September 2025. Date of publication 15 October 2025; date of current version 30 April 2026. This work was supported in part by Charles Sturt University and in part by the Australian Government Research Training Program (RTP) scholarship. This article was recommended for publication by Associate Editor Jingfeng Zhang upon evaluation of the reviewers' comments. *(Corresponding author: Naureen Naqvi.)*

Naureen Naqvi and Md Zahidul Islam with the School of Computing, Mathematics and Engineering, Charles Sturt University, Bathurst, NSW 2795, Australia (e-mail: nnaqvi@csu.edu.au; zislam@csu.edu.au).

Sabih ur Rehman is with the School of Computing, Mathematics and Engineering, Charles Sturt University, Port Macquarie, NSW 2444, Australia (e-mail: nnaqvi@csu.edu.au).

This article has supplementary downloadable material available at https://doi.org/10.1109/TAI.2025.3619461, provided by the authors.

Digital Object Identifier 10.1109/TAI.2025.3619461

2691-4581 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

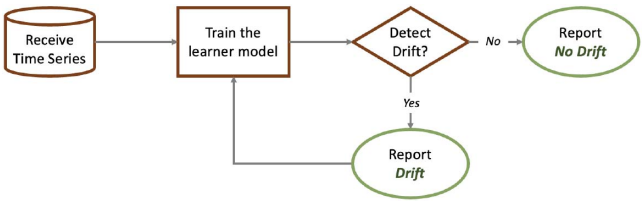


Fig. 1. Concept drift detection process using a learner model.

drift with minimal computational expenses. In the recent past, research efforts [21] have been placed in using an ensemble of existing univariate drift detection techniques to detect drift in multivariate data; however, our current effort is focused on using an ensemble of multiple distance test to accurately detect drift in univariate data.

To meet the criteria of an ideal drift detector, we previously introduced WinDrift (WD) [3], a time series-based method designed for efficient and accurate drift detection with low computational overhead. WD uses a two-sample approach, comparing time-based windows via the Kolmogorov–Smirnov (KS) test [22], which detects distributional changes through differences in empirical cumulative distribution functions (ECDFs). While KS was selected for its general robustness, relying on a single test may limit WD's effectiveness, particularly in real-world scenarios involving complex shifts in distributional characteristics such as mean, variance, and shape.

To address the limitations of WD, we introduce WinDrift++ (WD++), a novel and robust drift detection method. Like its predecessor, WD++ relies on two-sample tests between time-based windows but significantly enhances reliability by employing a committee of diverse statistical distance tests. These tests collectively contribute to a majority vote mechanism, improving the detector's sensitivity to a broader range of distributional changes. Inspired by ensemble learning in classification tasks [23], [24], WD++ harnesses the strengths of multiple tests to boost detection accuracy.

The proposed method, referred to as WD++, builds upon a previously introduced approach WD (outlined in Section II) but substantially extends it with key innovations, as detailed in Section III. Unlike WD as shown in Fig. 2, which used a single statistical test (KS test) [22], WD++ incorporates a suite of statistical tests, significantly improving robustness. We provide comprehensive experimental validation that demonstrates the advantages of WD++ over existing techniques.

WD++ is built for modularity, allowing users to tailor the drift detector by enabling or disabling specific statistical distance tests based on dataset characteristics. Users can also integrate alternative tests beyond those currently supported. By default, we recommend a core set of five widely adopted ECDF-based tests such as KS [22], Kuiper [25], Cramér–von Mises (CvM) [26], Anderson–Darling (AD) [27], and Wasserstein/earth mover distance (EMD) [28] as illustrated in Fig. 3(a). This ensemble captures diverse distributional shifts, enhancing robustness across varying drift scenarios.

The main contributions of this work are as follows.

1) We develop WD++, a reliable, robust, and efficient drift detector that uses multiple widely adopted statistical distance tests, as described in Sections II and III.
2) We evaluate WD++ against eight state-of-the-art drift detectors and WD, using 22 synthetic datasets and eight real-world datasets with known points of drift, as detailed in Section IV.
3) We release the development code and synthetic datasets in the final version of this article.

The structure of the article is as follows. Section II outlines the fundamental concepts, while Section III describes our proposed approach, including an algorithm and a step-by-step explanation. Section IV showcases the experimental findings, and Section V analyzes these results. Section VI summarizes the discussion and outlines potential future directions. Supplementary material includes further details on statistical methods, data generation processes, and computational analysis. A complete list of symbols and notations used throughout the article can be found in Table XI.

II. Preliminaries

This section introduces key concepts essential for understanding our method, as illustrated in Algorithm 1 and Figs. 2 and 3. We begin by explaining the ECDF statistical distance measure, fundamental to our work as given in (1). We then define the null hypothesis for our two-sample test as per (2) and Fig. 3(a). Finally, we provide a discussion of the ECDF-based statistical distance tests chosen for drift detection, as outlined in Section I.

In WD++, we use the ECDF to measure the distance between two time windows: the historical window W_H and the new data window W_N. We assume that both windows are drawn from the same population distribution P. The CalculateECDF operation, shown in Fig. 3(b), is detailed in Section A of the supplementary material. The ECDF for a single attribute X, with value x and n data records in a window, is defined as follows [29]:

$$\hat{F}_n(x) = \frac{\text{no. of data records in a window} \leq x}{n}. \quad (1)$$

A graphical representation of the ECDFs $\hat{F}_H(x)$ and $\hat{F}_N(x)$ is shown in Fig. 3(b) using blue and green lines, respectively. The null hypothesis H_0 of a two-sample test on a statistical distance metric is expressed as $D_s > D_c$, where D_s is the statistical distance between the ECDFs and D_c is the critical value, i.e., the maximum allowable threshold before the drift can be detected. However, since we use five statistical distance tests, the expressions for D_s and D_c vary for each test, as detailed in (3)–(7) and illustrated with red lines and shaded areas in Fig. 3(b). This hypothesis is given by the *AssessDrift* function in Algorithm 1

$$\left\{ \begin{array}{l} H_0 :\neq \text{drift},\ D_s < D_c \implies \{W_H, W_N\} \subseteq P \\ H_A := \text{drift},\ D_s > D_c \implies \{W_H, W_N\} \nsubseteq P \end{array} \right\}. \quad (2)$$

Next, we introduce the ECDF-based statistical distance tests used in WD++, which are widely adopted in the machine learning literature [30]. These tests calculate the distance between two distributions based on their ECDFs and assess the null hypothesis in (2). These statistical concepts are summarized in (3)–(7), with detailed explanations available in Section A of the supplementary material.

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

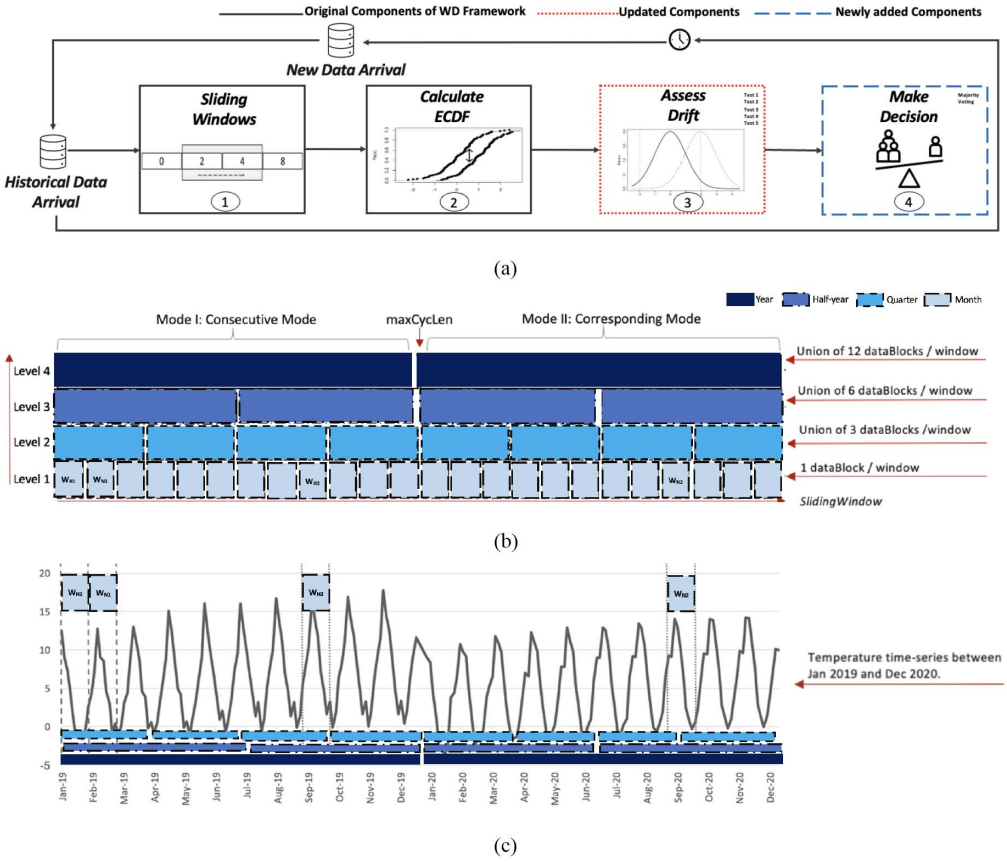


Fig. 2. WD++ operations: multiple sliding windows using dataBlocks and winLevSize using WD. (a) Schematic diagram of the WD++ workflow. (b) Multiple time-based hierarchical levels using winLevSize[ ] = {1, 3, 6, 12}. (c) Sample windows $\{W_H, W_N\}$ from time series data for drift detection operation.

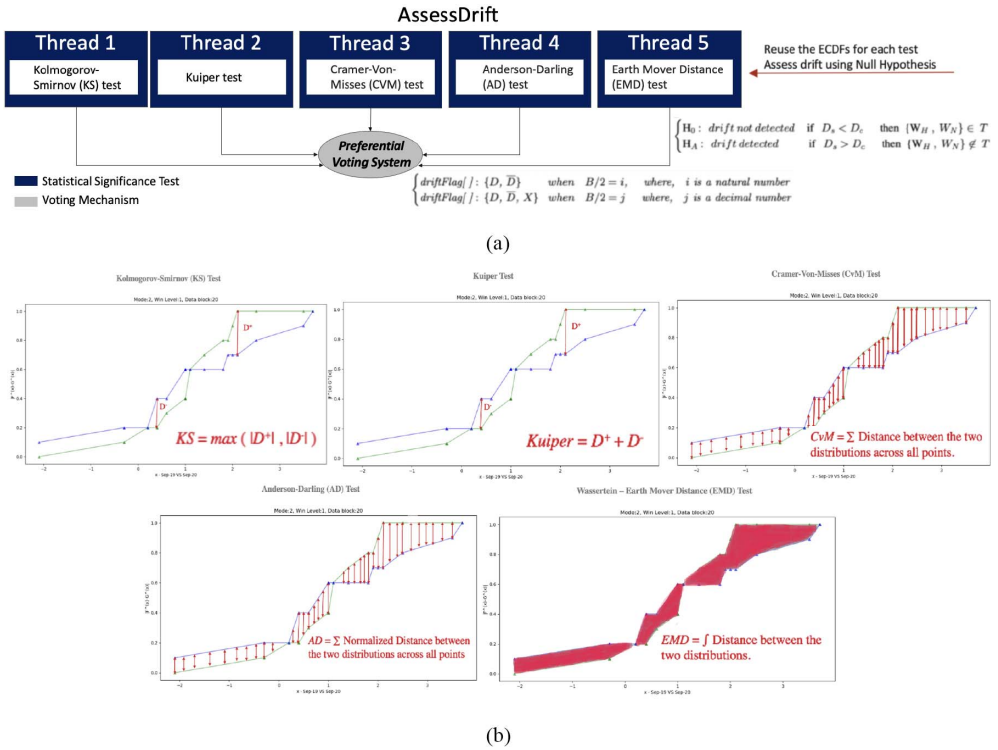


Fig. 3. WD++ operations: committee of statistical tests for drift detection. (a) AssessDrift between $\{W_H, W_N\}$ using statistical distances and majority voting. (b) Graphical representation of the five state-of-the-art statistical distance tests.

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

1) *KS test* [22] is mathematically expressed as follows:

$$\mathrm{KS} = \max_{x \in \mathbb{R}} |\hat{F}_H(x) - \hat{F}_N(x)| \tag{3}$$

where $\hat{F}_H(x)$ represents the ECDF of the historical window W_H, and $\hat{F}_N(x)$ shows the ECDF of the new data window W_N as given in Fig. 3(b).

2) The *Kuiper test* [25] is mathematically given as

$$\mathrm{Kuiper} = \max_{x \in \mathbb{R}}(\hat{F}_H(x) - \hat{F}_N(x)) \# x002B; \min_{x \in \mathbb{R}}(\hat{F}_H(x) - \hat{F}_N(x)). \tag{4}$$

3) The *CVM test* [26] is given as

$$\mathrm{CVM} = \sum_{x \in \mathbb{X}} |\hat{F}_H(x) - \hat{F}_N(x)|. \tag{5}$$

4) The *AD test* [27] is expressed as

$$\mathrm{AD} = \sum_{x \in \mathbb{X}} \frac{|\hat{F}_H(x) - \hat{F}_N(x)|}{\hat{C}(x)(1 - \hat{C}(x))} \tag{6}$$

where $\hat{C}(x)$ represents the ECDF of the combined distribution $W_{[H,N]}$ that consists of the ordered records from the combined set of data records in both the historical window W_H, and the new data window W_N.

5) The *EMD test* [28] is given as

$$\mathrm{EMD} = \int_{-\infty}^{\infty} |\hat{F}_H(x) - \hat{F}_N(x)| d(x). \tag{7}$$

Next, we present a detailed description of WD++.

Algorithm 1: WD++ Using Multiple ECDF-based Distance Tests.

Input: $winLevSize[]$, $dataBlocks[]$ (mandatory); α, $userContinue$ (optional).
Output: dF[ ].

$winlevsize[] \leftarrow \emptyset$ ▷ window sizes for drift assessment
$dataBlock[] \leftarrow \emptyset$ ▷ data used for drift assessment
$dBcount \leftarrow 0$ ▷ count of dataBlocks at a given time
$driftFlag[] \leftarrow \emptyset$ ▷ the outcome of drift assessment
$dBnum[] \leftarrow \emptyset$ ▷ W_{H1} (k_1), W_{N1} (l_1), W_{H2} (k_2), W_{N2} (l_2)
$userContinue \leftarrow 0$ ▷ allow user to continue to assess drift
Step 1: Get UserInput
while $dataBlock \neq \emptyset$ **do**
  get $userInput\{winlevsize[], \alpha, userContinue\}$
  get dataBlock ▷ arrival of data with a timestamp
  $i \leftarrow 0$ ▷ unit size of a window from winlevsize
  $j \leftarrow 0$ ▷ sliding window index for each level
  $m \leftarrow 0$ ▷ cycle count after maxCycLen
  $o \leftarrow 0$ ▷ no. of dataBlocks to reach maxCycLen
  $k_1 \leftarrow (i * j)$ ▷ W_{H1} for Mode I using historical data
  $l_1 \leftarrow (i * (j + 1))$ ▷ W_{N1} for Mode I using new data
  $k_2 \leftarrow (((m - 1) * o) + (k_1))$ ▷ W_{H2} for Mode II using historical data
  $l_2 \leftarrow ((m * o) + (k_1))$ ▷ W_{N2} for Mode II using new data
end while
Step 2: Assess Drift
$dF1 \leftarrow 0$ ▷ dataFlag received from AssessDrift using KS test
$dF2 \leftarrow 0$ ▷ dataFlag received from AssessDrift using Kuip test
$dF3 \leftarrow 0$ ▷ dataFlag received from AssessDrift using CvM test
$dF4 \leftarrow 0$ ▷ dataFlag received from AssessDrift using AD test
$dF5 \leftarrow 0$ ▷ dataFlag received from AssessDrift using EMD test
while $userContinue = true$ **do**
  return $dBnum[] \leftarrow \{k_1, l_1, k_2, l_2\}$ ▷ returns window indexes
  $dBcount \leftarrow (dBcount + 1)$ ▷ increment dataBlock count by 1
  SlidingWindow ▷ slides windows on each level using index
  CalculateECDF ▷ using Equation 1
  AssessDrift ▷ using Equation 2
    function $AssessDrift.KS(dF1, dBnum[])$ ▷ using Equation 3
    function $AssessDrift.Kuip(dF2, dBnum[])$ ▷ using Equation 4
    function $AssessDrift.CvM(dF2, dBnum[])$ ▷ using Equation 5
    function $AssessDrift.AD(dF4, dBnum[])$ ▷ using Equation 6
    function $AssessDrift.EMD(dF5, dBnum[])$ ▷ using Equation 7
end while
Step 3: Make a Decision
set $dF[] = \{dF1, dF2, dF3, dF4, dF5\}$ ▷ from all the tests.
function $Majority(dF[], dF1, dF2, dF3, dF4, dF5)$ ▷ voting method.
return $dF[]$

III. The WD++ Method

This section explains our method, highlighting the key components from Section III-A, along with a step-by-step guide in Section III-B. Readers are encouraged to refer to Fig. 2 for further clarification.

A. Key Components

Our proposed method, WD++ as presented in Fig. 2(a), builds on WD by using hierarchical and corresponding windows for drift detection, as shown in Fig. 2(b). A hierarchical window approach uses nested time windows (e.g., a year contains half-years, quarters, and months). Users can define the level of drift detection via the *winLevSize* input as illustrated in Fig. 2(c). Corresponding time windows enable users to compare two time windows (e.g., Q1 of two consecutive years) for detection by parsing the windows in *dataBlocks*, as explained in Section II-I-A1.

Comparing corresponding time windows can enhance concept adaptation and support data-driven decisions. However, early in a time series, sufficient historical data, such as from a second annual cycle, may be unavailable. To address this, a drift detector should support comparisons between consecutive windows until corresponding data becomes available. For instance, a user may need to assess drift between Q1 and Q2 of the same year when second-year data are missing. To provide this flexibility, both WD and WD++ offer two exclusive modes: mode I—consecutive, and mode II—corresponding. These modes operate independently and cannot run simultaneously. Once hierarchical time windows are initialized, drift detection follows the steps in Section III-A2, with final decisions made via majority voting (Section III-A3).

In WD++, drift decisions are made using a majority voting scheme across five statistical tests, each capturing different properties of the distribution, i.e., central tendency (e.g., KS and AD), dispersion (e.g., CVM), tail behavior (e.g., Kuiper), and overall divergence (e.g., EMD). This ensemble approach enhances robustness by minimizing the influence of limitations of any single test. In cases where test outcomes result in a tie (e.g., two tests detect drift, two reject it, and one is inconclusive), WD++ returns an "unsure" label to avoid misleading decisions. The "unsure" category plays a key role in real-world, noisy environments where weak or partial drift may occur. Instead of enforcing a binary classification, it highlights statistical disagreement and allows users to take a more thorough approach. Using this approach, the users can:

1) Inspect each test outcomes to identify the disagreement;
2) Incorporate domain knowledge or visual diagnostics; or
3) Adjust the voting threshold to increase confidence (e.g., requiring four out of five tests to agree).

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF THE FIVE STATISTICAL TESTS USED IN WD++ DRIFT DETECTION METHOD

Test	Main Sensitivity	Focus	Tail Sensitivity	Symmetry	Key Feature
KS	Max absolute difference in CDFs	Any region (central)	Low	No	Sensitive to large deviations in CDF
Kuiper	Sum of max positive & negative CDF deviations	Uniformly across CDF	Balanced	Yes	For cyclic data and tail-insensitivies
AD	Weighted squared difference in CDFs	Tails	High	No	More weight to tail differences
CVM	Integrated squared difference in CDFs	Whole distribution	Moderate	Yes	Emphasizes global shape differences
EMD	Minimal work to transform one dist. to another	Location & spread	Balanced	Yes	Geometric interpretation

This approach follows best practices in ensemble decision-making, prioritizing conservative action in uncertain areas over overfitting or false alarms. The strengths of each of these statistical tests are outlined in Table I.

The following key objectives guided the design, development, and testing of our drift detector.

1) *Objective #1: Reliable*: A drift detector that minimizes false positives (FPs) and false negatives (FNs), ensuring trust in its use for incremental learning in real-world applications.
2) *Objective #2: Robust*: A drift detector that remains effective under varying statistical characteristics in datasets, aiming for high detection accuracy.
3) *Objective #3: Efficient*: A detector that efficiently identifies drift in datasets with both sudden and gradual drift.

Next, we explain the following key components of *WD++*.

1) *Configure*: It configures multiple hierarchical windows.
2) *Assess*: It assesses drift using multiple statistical tests.
3) *Decide*: It makes a decision for drift using majority voting.

1) Configure: In WD++ as presented in Fig. 2(a), we address the diverse needs of real-world applications by allowing users to customize the frequency of drift detection, as illustrated in Fig. 2(b). For instance, a user analyzing temperature data as shown in Fig. 2(c) may prefer monthly drift checks, while other monitoring retail sales might opt for seasonal intervals. The following parameters are defined to support this adaptability.

a) *Number of window levels (winLevSize)*: This user-defined parameter specifies the hierarchical structure of time windows, where each higher level window encompasses one or more lower level windows. For example, a user may define *winLevSize* as $\{1, 3, 6, 12\}$, representing monthly, quarterly, half-yearly, and yearly windows, respectively. The specific configuration may vary depending on the application's requirements. As illustrated in Fig. 2(b), level 1 corresponds to the most granular detection level, i.e., monthly drift detection, while level 4 represents the most aggregated level, i.e., yearly drift detection.
b) *Maximum cycle length (maxCycLen)*: This parameter determines the duration over which drift detection is performed in consecutive mode before transitioning to corresponding mode. For instance, a *maxCycLen* of 12 months indicates that the system will assess drift between successive time windows for the first year. After this period, drift detection will be conducted between corresponding windows across cycles (e.g., comparing January 2022 with January 2023). This setting allows for flexible adaptation based on the availability of historical data and the temporal structure of the dataset.
c) *Minimum block size of data (dataBlock)*: This parameter specifies the smallest unit of data used to construct time windows at each hierarchical level. For example, at the monthly level, one dataBlock is sufficient to form a window. In contrast, higher level windows require aggregations of multiple dataBlocks, i.e., three for quarterly, six for half-yearly, and twelve for yearly windows. This flexible structure enables consistent and symmetric parsing of data across different temporal resolutions.
d) *Modes of drift detection*: The parameters maxCycLen and dataBlock play a strategic role in determining the mode of drift detection. WD++ supports two mutually exclusive modes, i.e., mode I: consecutive mode and mode II: corresponding mode. In mode I, drift is assessed between two consecutive windows at the minimum window level, as well as at all intermediate hierarchical levels, until the maxCycLen is reached. Once maxCycLen is achieved, the system transitions to mode II, where drift is evaluated between corresponding windows that are separated by one full cycle (e.g., comparing January of two consecutive years). Although mode II aligns with many real-world applications requiring periodic comparisons, it is often impractical at the early stages of data collection, especially in domains such as smart cities [1], [19], where sufficient historical data may not be immediately available. To address this, mode I allows early drift detection based on adjacent windows, enabling prompt response without waiting for complete cycles.

Furthermore, the hierarchical design of WD++ empowers users to operate at multiple temporal granularities, enhancing flexibility and control. It is recommended that the maxCycLen value be divisible by all underlying window sizes defined in winLevSize to ensure consistent and symmetric data partitioning. At each hierarchical level, windows are constructed using one or more units of dataBlock, as specified in the winLevSize configuration.

2) Assess: The second core component of *WD++* involves the application of five widely adopted ECDF-based statistical distance tests for drift detection, as illustrated in Fig. 3(a) and (b). While our default implementation uses this ensemble of tests, users retain the flexibility to modify the set of tests according to their specific application needs. The rationale behind selecting ECDF-based tests lies in their interpretability and the ability to visually inspect changes across data quantiles. The capabilities offered by the ECDF-based statistical tests are often more robust than the PDF-based statistical distance tests [31].

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

Each test independently produces a decision in the form of a *drift flag* (*dF*) from the set $\{D, \overline{D}\}$, indicating detected drift or no drift, respectively. These individual test decisions are then aggregated as inputs to the third component of our method, the *decide* component, which consolidates the final drift decision from the detections given by each statistical test.

One of the key objectives of *WD++* is to maintain computational efficiency through the use of adaptive forgetting strategies. In *mode I*, as new *dataBlocks* arrive, the ECDF $\hat{F}_N(x)$ is computed for the new window W_N and compared against the ECDF $\hat{F}_H(x)$ of the historical window W_H. Once the *maxCycLen* threshold is reached, ECDFs for all prior windows are recalculated, and the corresponding raw observations are discarded to reduce memory overhead.

In *mode II*, upon the arrival of new *dataBlocks*, *WD++* computes the ECDF $\hat{F}_N(x)$ of the current window W_N and compares it to the stored ECDF $\hat{F}_H(x)$ of the corresponding window from the previous cycle W_H. As the data are sequentially ordered, there is no need to retain timestamp-specific metadata. Only the result of the drift comparison is preserved, while all other data, including the ECDF from the previous cycle, are discarded after use. This strategy ensures memory efficiency without compromising analytical rigor.

3) Decide: In the third component, *dFs* are aggregated from each hierarchical level across all consecutive windows until the specified *maxCycLen* is reached. After this point, the collection of *dFs* continues for corresponding windows. These *dFs*, produced by the ensemble of five statistical tests described in the second component, serve as input to the majority voting mechanism for final drift decisions.

As illustrated in Fig. 2(a), consider an example with four hierarchical levels, i.e., monthly, quarterly, half-yearly, and yearly, and a *maxCycLen* of 12, representing the number of dataBlocks required to complete a full cycle. In consecutive mode, prior to reaching *maxCycLen*, drift is assessed between adjacent windows, yielding 11 *dFs* at the monthly level, three at the quarterly level, one at the half-yearly level, and none at the yearly level. Note that this is due to the fact that a full yearly cycle must be completed before a drift detection can be made. In corresponding mode, once *maxCycLen* is reached, comparisons are made between corresponding windows across cycles, producing 12 *dFs* at the monthly level, four at the quarterly level, two at the half-yearly level, and one at the yearly level.

Next, we define the majority voting mechanism and its associated concepts. In the majority voting process used in *WD++*, two key elements are required, i.e., the candidate set *B* and the set of votes *V* received by each candidate. In this context, the candidate set *B* consists of two possible outcomes: D for drift and $\overline{D}$ for no drift. The majority voting mechanism uses these inputs, i.e., *B* and *V*, to select the winning candidate, denoted as $W(B, V)$, which corresponds to the outcome that receives the most votes.

It is recommended that users select *B* to be an odd number, ensuring two possible outcomes, i.e., $\{D, \overline{D}\}$. However, if *B* is even, the detector will yield three possible outcomes, i.e., $\{D, \overline{D}, U\}$, where U stands for unsure. The unsure outcome occurs when there is a tie between the votes for $\{D, \overline{D}\}$. In such cases, it is at the user's discretion to interpret the result as either D or $\overline{D}$ depending on their desired confidence level.

To detect concept drift, WD++ employs five statistical distance tests (3)–(7) in parallel, each capturing different aspects of distributional change (e.g., shifts in mean, variance, and shape). This multitest approach provides a robust and comprehensive drift assessment, leveraging the strengths and sensitivities of individual tests. The rationale for selecting these tests is detailed in Section I, the working details provided in Section II, with their characteristics compared in Table I. The framework uses a majority voting method across the five tests but is designed to be modular, which means that the users can enable or disable specific tests based on their application needs.

B. Step-by-Step Description

Using Algorithm 1 and Fig. 2, we present a step-by-step guide for the WD++ method, outlined as follows.

1) *Step 1: Get UserInput*: It initializes input and variables.
2) *Step 2: Assess Drift*: The hierarchical and corresponding time windows are prepared using the same approach as WD, and the drift is assessed using five statistical distance tests. The drift detection results for each test are collected.
3) *Step 3: Make a Decision*: A decision is made on drift detection by taking a majority vote on the detection results received from step 2.

A detailed explanation of these steps is provided as follows.

1) Step 1: Get User Input: In the first step, we gather two types of inputs, i.e., mandatory and optional inputs. The mandatory inputs include the *winLevSize* and *dataBlocks*, which are detailed in Section III-A. The optional input is the α parameter, which is used for calculating the critical values for each statistical test, as explained in Section II. By default, α is set to 0.050, corresponding to a 95% confidence interval (CI). However, users can adjust this value according to their needs. The drift detection results, referred to as *dFs*, are output by the algorithm as binary values: 1 denotes *D* (drift detected), while 0 denotes $\overline{D}$ (no drift detected).

2) Step 2: Assess Drift: In this step, windows are generated for each level of the hierarchy and across both mode I: consecutive mode and mode II: corresponding mode, based on the winLevSize parameter. The windows are organized separately at each hierarchical level to ensure accurate drift detection between the correct pairs of windows. In mode I: consecutive mode, the windows are represented as pairs (W_{H1}, W_{N1}), where W_{H1} is the historical window and W_{N1} is the new window. In mode II: corresponding mode, the windows are represented as pairs (W_{H2}, W_{N2}), where W_{H2} and W_{N2} represent the historical and new windows, respectively, separated by one full cycle. For each pair of windows, drift is evaluated using all five statistical distance tests, as specified in (3)–(7). The results from each test are recorded and stored separately for further analysis.

3) Step 3: Make a Decision: In this step, a final decision is made based on a majority voting scheme applied to the drift detection results obtained from each statistical test. The drift detection outcomes from each hierarchical level, across both mode I: consecutive mode and mode II: corresponding mode

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

are aggregated. The results are then reported as received from each test, with a decision made according to the majority of votes received through the voting mechanism.

IV. EXPERIMENTAL RESULTS

This section is organized into three sections: Section IV-A provides an overview of our experimental setup; Section IV-B introduces both the synthetic and real datasets used in the study; and Section IV-C presents a detailed analysis of the results. For completeness, additional results are presented in Section B of the supplementary material.

A. Experimental Setup

All experiments were performed on a machine with 12 cores[1] and 32.0 GB of RAM. The coding of *WD++* was carried out using PyCharm IDE 2021.2 [32]. Each experiment was conducted five times, with a 95% CI. *WD++* was evaluated against nine competitive drift detection methods. These methods include WD [3], drift detection method based on Hoeffding's bounds with moving average-test (HDDMA), drift detection method based on McDiarmid's bounds with exponentially weighted moving average (HDDMW) [9], early drift detection method (EDDM) [33], adaptive WINdowing (ADWIN) [34], Page Hinkley test (PHT) [35], KSWin [36], reactive drift detection method (RDDM) [37], and EWMA for concept drift detection (ECDD-WT) [38]. All detectors were configured with the default parameter values as specified in the Scikit multiflow framework [39] and the Frouros Python library [40].

We assess various drift detection metrics to evaluate the performance of *WD++* in comparison to other state-of-the-art detectors. These metrics include recall, precision, true positive rate (TPR), false positive rate (FPR), true negative rate (TNR), false negative rate (FNR), Matthews correlation coefficient (MCC), and F_1 score, as presented in literature [41]. Additionally, we consider mean time to false alarms (MTFAs), mean detection rate (MDR), mean time to detection (MTD), and mean time to true detection (MTTD), as discussed in the literature review of evaluation metrics [42]. For an indirect measure of the performance of a drift detector, it can be combined with a classifier to evaluate improvements in classification accuracy when the detector is used [43]. We compare the performance of *WD++* with nine drift detectors across all of these metrics.

In our experiments with synthetic datasets containing known drift points, we follow a standard evaluation process where only one true positive (TP) detection is allowed per actual concept drift. A detection is considered a TP if it occurs within a predefined detection delay window (e.g., 200 records) after the true drift point. Any additional detections within this window are disregarded to prevent inflation of the TP count. Detections outside this window are categorized as either FPs or FNs, depending on the context. This approach ensures consistency and comparability across all evaluated drift detection methods.

[1]Intel(R) Xeon(R) CPU i7-9750H 2.60GHz, 64-bit OS, x64-based processor.

B. Datasets

This study evaluates WD++ using 22 synthetic and eight real-world datasets, grouped into univariate synthetic dataset (USD), univariate real dataset (URD), multivariate synthetic dataset (MSD), and multivariate real dataset (MRD) categories. These datasets (100–150 MB on average) were selected to represent diverse drift types, including shifts in mean, variance, skewness, kurtosis, multimodality, and tail behavior. Synthetic datasets (Sections IV-B1 and IV-B4) offer controlled ground truth drift points, while real-world datasets (Sections IV-B2 and IV-B3) include documented drift periods aligned with real events. Note that in our experiments using univariate data, a Naive Bayes classifier is incrementally trained on a single feature stream and its binary label. Each concept dataset represents a distinct data distribution. Further details on dataset are available in Section B of the supplementary material.

1) USDs: To rigorously evaluate our method, we designed 17 USDs that simulate both drift and no-drift scenarios using data from two consecutive years, Y2019 and Y2020, as introduced in Section III. This design enables experimentation under clearly defined statistical settings and supports reproducible benchmarking for concept drift detection. The synthetic datasets were generated using normal distributions with randomly sampled means (μ) and standard deviations (σ) for Y2019, which serves as the reference distribution. Variations introduced in Y2020 allow us to simulate different forms of distributional drift. As outlined in Section IV-B1, the datasets are organized across three distinct experimental setups:

1) *Experiment 1*: Varying μ and σ such that a similar distribution is generated for both Y2019 and Y2020 to confirm no location-based drift.
2) *Experiment 2*: Varying μ and σ such that the inverse distribution is generated for both Y2019 and Y2020 to confirm location-based drift.
3) *Experiment 3*: Simultaneously varying both μ and σ to capture combined drift scenarios including skewness, outliers, ties, and kurtosis.

Each experiment comprises a mixture of drift and no-drift cases to test the sensitivity and robustness of our method under diverse change conditions. Further technical details and illustrative examples are provided in Figs. 7–9 in the supplementary material.

a) Experiment 1: Similar datasets: In experiment 1, we generated five datasets (C_1 to C_5), where the statistical properties between Y2019 and Y2020 are intentionally kept similar, simulating no-drift scenarios. Each dataset is derived from a normal distribution, with expressions provided in Table II and illustrated in Fig. 7(a). For instance, dataset C_1 begins with $\mu_1 = 5.00°C$ and $\sigma_1 = 1.00°C$. In subsequent months, μ increases by 1.25°C and σ by 0.25°C, resulting in a range from 5 to 18.75°C. Both Y2019 and Y2020 use identical parameters per month, as visualized in Fig. 8(a) and (b), confirming the statistical similarity across years.

This setup allows us to evaluate how well different methods avoid FPs. As discussed in Section III, we apply four levels of window sizes (*winLevSize* = 1, 3, 6, 12) corresponding to

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUMMARY OF UNIVARIATE SYNTHETIC DATASETS (C_1–C_{17})

Pattern	Desc.	Value/ Change Parameter
C_1, C_6	μ_1	Initially set
	σ_1	Initially set
C_2, C_7	μ_2; σ_2	$\mu_1 - 10\%$; σ_1 - 10%
C_3, C_8	μ_3	$\mu_1 - 20\%$
	σ_3	$\sigma_1 - 20\%$
C_4, C_9	μ_4	$\mu_1 + 10\%$
	σ_4	$\sigma_1 + 10\%$
C_5, C_{10}	μ_5	$\mu_1 + 20\%$
	σ_5	$\sigma_1 + 20\%$
C_{11}	Skewness$*$	Median and standard deviation.
C_{12}	Shape$*$	Standard deviation.
C_{13}	Location$*$	Mean.
C_{14}	Light tail$**$	Less deviation from the mean.
C_{15}	Heavy tail$**$	More deviation from the mean.
C_{16}	Outliers$**$	Extreme values.
C_{17}	Ties$**$	Repetitive values.

Note: μ_1 = 5.00, and σ_1 = 1.00; Desc. = Description of the statistical characteristics that have been changed in the dataset. Value for C_1–C_{10} = shows how it is calculated; Change parameter for C_{11}–C_{17} = Indicates the parameter that has been changed to achieve a change in the overall statistical characteristics of a distribution. $*$ = significant change; $**$ = insignificant change. The definition of "significant/ insignificant change" is explained in Section B-1 of the supplementary materials.

monthly through yearly detection granularity. Using the KS test on ECDFs, we compare historical windows (W_H) to new data windows (W_N), with results shown in Fig. 8(a) and (b). While drift is detected within 2019 (e.g., January and February), no drift is found between identical months of 2019 and 2020, validating the test case design.

b) Experiment 2: Dissimilar datasets: In experiment 2, we constructed five datasets (C_6 to C_{10}) to simulate drift scenarios by introducing distributional dissimilarity between Y2019 and Y2020. The statistical expressions used are listed in Table II, and sample distributions are visualized in Fig. 7(b).

Here, Y2019 retains the reference distributions from Section IV-B1. To create dissimilarity, the monthly parameters (μ, σ) from Y2019 are reversed when generating Y2020. Specifically, the values for December 2019 are used for January 2020, November for February, and so on, as illustrated in Fig. 8(c) and (d). This reversal induces drift between corresponding periods at all levels (monthly, quarterly, etc.), making it an ideal setup to test sensitivity to distributional changes.

c) Experiment 3: Change in statistical characteristics: This experiment evaluates the ability of WD++ to detect drift caused by changes in statistical characteristics rather than full distributional shifts. We generated seven datasets (C_{11} to C_{17}) by modifying the baseline Y2019 distribution [Fig. 9(a)] to create altered Y2020 distributions [Fig. 9(b)–(h)], adjusting parameters such as mean, standard deviation, skewness, tailedness, and outliers (see Table II). In these figures, each value of x represents an observation from the domain of a normally distributed random variable X, corresponding to a point along the horizontal axis of the bell-shaped curve. Unlike prior experiments, drift here is not always visually apparent. We applied targeted significance tests such as T-test, F-test, Wilcoxon Rank-Sum, and others, to determine whether a change is statistically significant and thus labeled as drift. This reflects realistic scenarios where subtle but important changes occur across multiple characteristics. For example, in C_{12} [Fig. 9(c)], we gradually increased the standard deviation until the change became statistically significant. While we primarily modified one parameter per dataset, secondary effects (e.g., tailedness shifts in C_{14} and C_{15}) are acknowledged. Datasets C_{11} to C_{13} represent pronounced changes; C_{14} to C_{17} involve subtler adjustments. This setup enables comprehensive evaluation of both drift and no-drift scenarios.

2) URDs: We evaluated *WD++* on five real-world univariate datasets, illustrated in Fig. 4. The corresponding time series line charts exhibit observable shifts in data patterns, often indicative of drift. To establish a ground truth, we cross-referenced detected drift with well-documented historical events from the literature. However, it is essential to recognize that drift may also occur outside these predefined intervals, particularly in real-world data streams.

Drift detection in real-world datasets poses unique challenges due to noisy signals, gradual changes, and overlapping factors. To thoroughly assess *WD++*, we selected real datasets previously shown to exhibit concept drift, guided by a targeted literature review spanning multiple domains. Each case represents a shift in the underlying distribution due to a significant event. The historical context is as follows.

1) C_{18}: A surge in international arrivals in 1990 due to the Commonwealth Games [Fig. 10(a)] [44].
2) C_{19}: A rise in accommodation occupancy rates during the 2003 Triathlon World Championship [Fig. 10(b)] [45].
3) C_{20}: A spike in mortality in 2011 following the Christchurch earthquake [Fig. 10(c)] [46].
4) C_{21}: An increase in unemployment rates during the global financial crisis (GFC) (2007–2009) [Fig. 10(d)] [47].
5) C_{22}: A rise in GDP growth between 2010 and 2012 attributed to the Australian mining boom [Fig. 10(e)] [48].

These examples validate the applicability of *WD++* across diverse domains and confirm its effectiveness in detecting drift even under complex real-world conditions.

3) MRDs: To assess WD++ on multivariate data streams, we used several benchmark datasets commonly cited in drift detection research (see Table III). Although our approach is univariate by design, we extend it to multivariate settings by applying the detector to each variable independently and averaging the results. This allows for fair and scalable evaluation in high-dimensional contexts. The key characteristics of these real-world datasets offer a comprehensive testbed to evaluate robustness and adaptability.

a) *Nebraska weather prediction dataset* [2]: This dataset, from the U.S. National Oceanic and Atmospheric Administration (NOAA), spans 50 years of daily meteorological measurements. It includes 18 159 records, eight attributes (e.g., temperature, pressure, and wind speed), and two class labels, offering insights into long-term weather patterns.
b) *Covertype dataset* [49]: Collected by the U.S. Forest Service, this dataset describes forest cover types across 30×30 meter grid cells. It comprises 581 012 records,

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

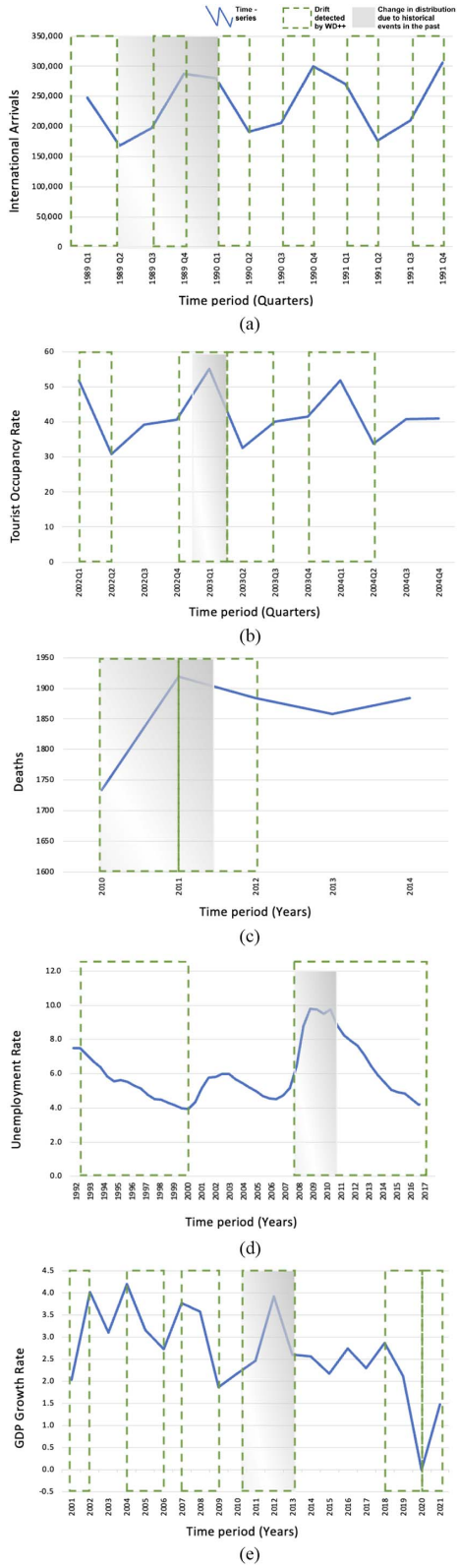


Fig. 4. Visual representation of the five univariate real datasets (C_{18}–C_{22}). (a) *Tourism* (New Zealand): drift in New Zealand's international arrivals rates in 1990 due to the first Commonwealth games (C_{18}). (b) *Accommodation* (Otago, NZ): drift in New Zealand's Tourist occupancy rate in 2003 due to the first Triathlon championship (C_{19}). (c) *Disaster impact* (New Zealand): drift in New Zealand's number of deaths in 2011 due to an earthquake in Christchurch (C_{20}). (d) *Unemployment* (USA): drift in the unemployment rates in the United States in 2007–2009 due to the GFC (C_{21}). (e) *Mining boom* (Australia): drift in Australia's gross domestic product (GDP) growth in 2010–2012 due to the mining boom (C_{22}).

TABLE III
SUMMARY OF MULTIVARIATE REAL AND SYNTHETIC DATASETS ($C_2$3–C_{30})

Pattern	Name	T	# Cl.	# Att.	# Records	Acc. (%)
C_{23}	NOAA	R	2	8	18 159	69.74
C_{24}	Covertype	R	7	54	581 012	48.76
C_{25}	Electricity	R	2	8	45 312	57.41
C_{26}	Insects	R	6	33	52 848	16.07
C_{27}	SEA_a	S	2	3	1 000 000	57.55
C_{28}	SEA_g	S	2	3	1 000 000	57.55
C_{29}	AGR_a	S	2	9	1 000 000	52.83
C_{30}	AGR_g	S	2	9	1 000 000	52.83

Note: Acc. stands for Accuracy of Majority class label (%); T stands for data type; R stands for Real-time dataset; S stands for Synthetic dataset; a stands for Abrupt drift and g stands for Gradual drift. # Cl. is the number of classes. # Att. is the number of attributes.

54 attributes (numeric and categorical), and seven class labels, making it a high-dimensional and heterogeneous dataset.

c) *Electricity dataset* [50]: Originated from the New South Wales Electricity Market, it contains 45 312 records with eight attributes and two class labels (UP and DOWN). The data reflect electricity price movements influenced by supply-demand dynamics, with a 5-min sampling interval.

d) *INSECTS dataset* [51]: Captured via optical sensors in nonstationary environments, this dataset monitors insect by recording environmental factors such as temperature, humidity, and luminosity. It consists of 52 848 records, 33 attributes, and six class labels and is specifically designed for evaluating drift in streaming classification tasks.

4) MSDs: To evaluate our method in controlled settings, we used two standard synthetic data stream generators that allow precise manipulation of drift dynamics (see Table III). Each stream ran for one million time steps to ensure consistency and computational efficiency. Drifts were introduced by altering data-generating functions, following the method in [36], with both abrupt and gradual drifts simulated via midstream rule changes. While this study focuses on these two drift types, future work will explore more complex scenarios such as fast and moderate incremental drifts. Details of the two generators are provided as follows.

a) The SEA generator [52] produces data with three continuous attributes (f_1, f_2, f_3), each ranging from 0 to 10. Only f_1 and f_2 are relevant to the classification task; and f_3 is irrelevant. Class labels are determined using simple threshold-based rules in a 2-D space spanned by f_1 and f_2. The space is divided into four blocks, each defined by a different threshold θ: *block 1*: $\theta = 8$; *block 2*: $\theta = 9$; *block 3*: $\theta = 7$; and *block 4*: $\theta = 9.5$. A point is assigned to class 1 if $f_1 \# x002B; f_2 \leq \theta$, and to class 0 otherwise. The generator supports a default noise factor of 10%. We use two variations: SEA_a: it introduces three abrupt drifts; and SEA_g: it introduces three gradual drifts.

b) The AGRAWAL (AGR) generator [53] simulates credit approval scenarios using ten predefined classification functions. It generates streams with three continuous and six nominal attributes. Data are labeled into two classes, and values are sampled from uniform distributions. We follow the configuration used in [54], including a 10%

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

noise to simulate real-world uncertainty. AGR_a simulates three abrupt drifts. AGR_g simulates three gradual drifts.

C. Results

In this section, we present the results of our experiments designed to evaluate the effectiveness of *WD++* in detecting concept drift across various scenarios. We assess its performance on both synthetic and real-world datasets, spanning univariate and multivariate domains. These experiments are structured to test the method under controlled drift conditions (no drift, abrupt, and gradual), as well as in realistic environments with historical ground truth. We compare our approach against five established drift detectors using metrics such as detection accuracy, FPR, detection delay, and indirect measure of detection accuracy through classification accuracy. The results are organized as follows: we first discuss performance on USDs (Section IV-C1), followed by real-world univariate datasets (Section IV-C2), and finally, we analyze multivariate settings (Sections IV-C3 and IV-C4). This structure allows for a comprehensive evaluation of the robustness, reliability, and generalizability of our method.

1) USDs: This section presents the results of our experiments conducted on 17 synthetic datasets, spanning experiments 1, 2, and 3 (see IV-B1)). These datasets were constructed to evaluate drift detection performance under a wide range of conditions, including identical, dissimilar, and statistically altered distributions. Across all synthetic datasets, WD++ consistently achieved the highest performance in terms of detection delay as can be seen in Tables IV, drift detection accuracy as shown in Table Tables IV and Tables V and resource utilization as shown in Table Tables VI and Tables VII demonstrating its robustness and responsiveness across varied drift scenarios.

a) Experiment 1: Similar datasets: As outlined in Section IV-B1, experiment 1 is designed to evaluate detector behavior under *no drift* conditions. In such scenarios, it is expected that a reliable drift detector consistently identifies the absence of drift. As shown in Table Tables IV and V, only *WD*, *WD++*, and EDDM among the five evaluated methods successfully avoided false detections across all datasets. Notably, *WD++* exhibited the most consistent performance, emphasizing its *reliability*, a key criterion discussed in Section III-A.

We substantiate these results through empirical validation. Recall from Section III that we evaluate drift across four temporal resolutions using *winLevSize* values of 1, 3, 6, and 12, corresponding to monthly, quarterly, semiannual, and annual comparisons, respectively. Fig. 8(a) and (b) presents ECDF plots for C_1–C_5 at the monthly level. In each plot, the green curve represents the empirical distribution of the historical window (W_H), the blue curve corresponds to the new window (W_N), and the red vertical line marks the KS distance between the two distributions. Fig. 8(a) illustrates a rare case of apparent drift between January and February 2019, while Fig. 8(b) confirms stability between January 2019 and January 2020, aligning with the quantitative findings in Table IV and validating the absence of drift.

This experiment is particularly important for evaluating the *FPR* of each method. As emphasized in Section I, minimizing false alarms is crucial in real-world applications, where unwarranted drift may disrupt the learning process or trigger unnecessary model updates. The results show that *WD++* achieves the lowest FPR, reinforcing its robustness and dependability in stable environments.

b) Experiment 2: Dissimilar datasets: As described in Section IV-B1, the second experiment focuses on scenarios where distributional dissimilarity is deliberately introduced. Datasets C_6–C_{10} are constructed by inverting the statistical characteristics between corresponding periods. Given this setup, all drift detectors are expected to correctly identify significant changes between the respective periods. As shown in Tables IV, all detectors except EDDM successfully detected drift across these datasets. This highlights the limitations of EDDM in handling clearly defined dissimilar distributions.

To complement the quantitative results, we performed a visual analysis using the sample KS test, as defined in (3). Fig. 8(c) and (d) presents ECDF comparisons at the monthly level. For instance, Fig. 8(c) illustrates a clear drift between January and February 2019, while Fig. 8(d) shows a significant change between January 2019 and January 2020. These findings are consistent with our tabular results and provide visual confirmation of the distributional shifts. Compared to the baseline WD method, our proposed approach WD++ consistently achieves higher detection accuracy across all dissimilar datasets. Notably, WD++ records the lowest number of FNs, demonstrating its capability to detect meaningful changes effectively. This is important in real-world applications, where missed drifts can lead to outdated or suboptimal models. Thus, WD++ demonstrates both *reliability* and *robustness* in data evolution in real-time environments.

c) Experiment 3: Change in statistical characteristics: As outlined in Section IV-B1, this experiment evaluates drift detection when the underlying statistical characteristics of the distributions are selectively altered. The reference distributions from Y2019 are depicted in Fig. 9(a), while the modified distributions for Y2020, corresponding to datasets C_{11}–C_{17}, are illustrated in Fig. 9(b)–(h). Among these, datasets C_{11}–C_{13} feature significant distributional changes, whereas C_{14}–C_{17} incorporate only minor modifications, as summarized in Table II.

The primary aim of this experiment is to assess whether our proposed method is sufficiently *reliable* and *robust* to detect both drift and no-drift scenarios across varying levels of statistical change. Tables IV and V compare the performance of WD++ with several state-of-the-art detectors across all seven datasets. The results reveal that all detectors successfully identify drift in cases involving substantial statistical changes. However, only WD and WD++ accurately recognize the absence of drift in datasets with minor modifications (C_{14}–C_{17}). Importantly, WD++ demonstrates a clear improvement over its predecessor, WD, by reducing FPs and achieving more balanced detection outcomes.

This enhanced accuracy is largely attributed to the ensemble nature of our approach, which integrates multiple statistical

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
CONFUSION MATRIX WITH KEY DRIFT DETECTION PERFORMANCE METRICS

(a) Results on USD (C_1–C_5)

Methods	TP	TN	FP	FN	FPR	FNR	Acc.	Rec.	Prec.	F_1	MCC	MTFA	MDR	MTD	MTTD
ADWIN	0	0	37	0	1.00	-	0	-	0	-	-	1.00	-	1.79	-
EDDM	0	0	4	0	1.00	-	0	-	0	-	-	1.00	-	3.16	-
$HDDM_A$	0	0	605	0	1.00	-	0	-	0	-	-	1.00	-	0.13	-
$HDDM_W$	0	0	410	0	1.00	-	0	-	0	-	-	1.00	-	0.17	-
PHT	0	0	37	0	1.00	-	0	-	0	-	-	1.00	-	2.41	-
WD	0	0	3	0	1.00	-	0	-	0	-	-	1.00	-	2.06	-
WD++	0	0	0	0	1.00	-	-	-	-	-	-	1.00	-	-	-
KSWin	0	0	407	0	1.00	-	0	-	0	-	-	1.00	-	0.12	-
RDDM	0	0	353	0	1.00	-	0	-	0	-	-	1.00	-	0.20	-
ECDD-WT	0	0	281	0	1.00	-	0	-	0	-	-	1.00	-	0.34	-

(b) Results on USD (C_6–C_{10})

Methods	TP	TN	FP	FN	FPR	FNR	Acc.	Rec.	Prec.	F_1	MCC	MTFA	MDR	MTD	MTTD
ADWIN	19	0	26	0	1.00	0	0.43	1.00	0.43	0.60	-	0.57	0.00	1.66	3.42
EDDM	5	0	0	0	1.00	0	1.00	1.00	1.00	1.00	-	0.09	0.00	11.74	9.40
$HDDM_A$	19	0	598	0	1.00	0	0.03	1.00	0.03	0.06	-	0.97	0.00	0.13	3.37
$HDDM_W$	19	0	408	0	1.00	0	0.04	1.00	0.04	0.09	-	0.96	0.00	0.20	3.63
PHT	19	0	16	0	1.00	0	0.55	1.00	0.55	0.71	-	0.45	0.00	2.83	4.47
WD	18	0	0	0	1.00	0	1.00	1.00	1.00	1.00	-	0.01	0.00	0.73	0.56
WD++	18	0	0	0	1.00	0	1.00	1.00	1.00	1.00	-	0.01	0.00	0.82	0.44
KSWin	19	0	480	0	1.00	0	0.04	1.00	0.04	0.07	-	0.96	0.00	0.16	3.53
RDDM	19	0	387	0	1.00	0	0.05	1.00	0.05	0.09	-	0.95	0.00	0.23	4.42
ECDD-WT	19	0	291	0	1.00	0	0.06	1.00	0.06	0.12	-	0.94	0.00	0.19	2.53

(c) Results on USD (C_{11}–C_{17})

Methods	TP	TN	FP	FN	FPR	FNR	Acc.	Rec.	Prec.	F_1	MCC	MTFA	MDR	MTD	MTTD
ADWIN	8	0	0	7	-	0.47	0.53	0.53	1.00	0.70	-	0.00	0.47	2.94	8.13
EDDM	0	0	0	18	1.00	1.00	0.00	0.00	0.00	-	-1.00	1.00	1.00	-	-
$HDDM_A$	15	0	433	0	1.00	0.00	0.03	1.00	0.03	0.06	-	0.97	0.00	0.17	4.27
$HDDM_W$	15	0	255	0	1.00	0.00	0.06	1.00	0.06	0.11	-	0.94	0.00	0.31	4.60
PHT	15	0	16	0	1.00	0.00	0.49	1.00	0.49	0.66	-	0.51	0.00	3.19	5.67
WD	15	0	1	0	1.00	0.00	0.94	1.00	0.94	0.97	-	0.06	0.00	1.44	0.67
WD++	15	0	1	0	1.00	0.00	0.94	1.00	0.94	0.97	-	0.06	0.00	1.44	0.67
KSWin	15	0	487	0	1.00	0.00	0.03	1.00	0.03	0.06	-	0.97	0.00	0.16	4.47
RDDM	15	0	475	0	1.00	0.00	0.03	1.00	0.03	0.06	-	0.97	0.00	0.19	5.60
ECDD-WT	15	0	411	0	1.00	0.00	0.04	1.00	0.04	0.07	-	0.96	0.00	0.14	3.20

(d) Results on URD (C_{18}–C_{22})

Methods	TP	TN	FP	FN	FPR	FNR	Acc.	Rec.	Prec.	F_1	MCC	MTFA	MDR	MTD	MTTD
ADWIN	23	0	0	8	-	0.26	0.74	0.74	1.00	0.85	-	-	0.26	3.22	2.83
EDDM	0	0	31	31	1.00	1.00	0.00	0.00	0.00	-	-1.00	1.00	1.00	-	-
$HDDM_A$	7	0	0	24	-	0.78	0.22	0.22	1.00	0.37	-	-	0.78	11.14	9.14
$HDDM_W$	9	0	0	22	-	0.71	0.29	0.29	1.00	0.46	-	-	0.83	18.85	16.35
PHT	5	0	0	26	-	0.83	0.17	0.17	1.00	0.29	-	-	0.83	18.85	16.35
WD	28	0	0	3	-	0.10	0.90	0.90	1.00	0.95	-	-	0.10	0.47	0.36
WD++	31	0	0	0	-	0.01	0.99	0.99	1.00	0.99	-	-	0.01	0.49	0.26
KSWin	26	0	0	5	-	0.17	0.83	0.83	1.00	0.91	-	-	0.17	3.00	2.58
RDDM	20	0	0	11	-	0.35	0.65	0.65	1.00	0.79	-	-	0.35	4.55	4.16
ECDD-WT	17	0	0	14	-	0.46	0.54	0.54	1.00	0.70	-	-	0.46	3.55	2.89

(e) Results on MRD (C_{23}–C_{26})

Methods	TP	TN	FP	FN	FPR	FNR	Acc.	Rec.	Prec.	F_1	MCC	MTFA	MDR	MTD	MTTD
ADWIN	8	0	38	0	1.00	0.00	0.18	1.00	0.18	0.30	-	1.21	0.00	1.60	7.98
EDDM	1	0	0	7	-	0.88	0.13	0.13	1.00	0.22	-	-	0.88	74.00	47.00
$HDDM_A$	8	0	227	0	1.00	0.00	0.03	1.00	0.03	0.07	-	1.04	0	0.34	8.00
$HDDM_W$	8	0	213	0	1.00	0.00	0.04	1.00	0.04	0.07	-	1.04	0	0.40	8.63
PHT	8	0	30	0	1.00	0.00	0.21	1.00	0.21	0.35	-	1.27	0	3.32	10.63
WD	7	0	0	1	-	0.13	0.88	0.88	1.00	0.93	-	-	0.13	1.58	1.43
WD++	8	0	0	0	-	0.00	1.00	1.00	1.00	1.00	-	-	0.00	2.18	1.00
KSWin	8	0	153	0	1.00	0.00	0.05	1.00	0.05	0.09	-	1.05	0	0.45	8.38
RDDM	8	0	204	0	1.00	0.00	0.04	1.00	0.04	0.07	-	1.04	0	0.41	10.50
ECDD-WT	8	0	217	0	1.00	0.00	0.04	1.00	0.04	0.07	-	1.04	0	0.25	6.00

(f) Results on MSD (C_{27}–C_{30})

Methods	TP	TN	FP	FN	FPR	FNR	Acc.	Rec.	Prec.	F_1	MCC	MTFA	MDR	MTD	MTTD
ADWIN	8	0	26	0	1.00	0.00	0.24	1.00	0.24	0.39	-	1.31	0.00	2.18	7.98
EDDM	1	0	0	7	-	0.88	0.13	0.13	1.00	0.22	-	-	0.88	74.00	47.00
$HDDM_A$	8	0	197	0	1.00	0.00	0.04	1.00	0.04	0.08	-	1.04	0.00	0.40	8.00
$HDDM_W$	8	0	34	0	1.00	0.00	0.19	1.00	0.19	0.32	-	1.24	0.00	2.47	8.63
PHT	8	0	27	0	1.00	0.00	0.23	1.00	0.23	0.37	-	1.29	0.00	3.60	10.63
WD	7	0	0	1	-	0.13	0.88	0.88	1.00	0.93	-	-	0.13	-	1.14
WD++	8	0	3	0	1.00	0.00	0.73	1.00	0.73	0.84	-	3.67	0.00	4.33	1.25
KSWin	8	0	117	0	1.00	0.00	0.06	1.00	0.06	0.12	-	1.07	0.00	0.62	8.38
RDDM	8	0	155	0	1.00	0.00	0.05	1.00	0.05	0.09	-	1.05	0.00	0.56	10.50
ECDD-WT	8	0	145	0	1.00	0.00	0.05	1.00	0.05	0.10	-	1.06	0.00	0.38	6.00

distance measures to evaluate distributional changes. These findings reinforce our claim that WD++ offers improved robustness and reliability in diverse data environments, including those with subtle or borderline drift conditions.

2) URDs: This section presents results from five real-world datasets (C_{18}–C_{22}), each with known or hypothesized drift events (see Section IV-B2). As shown in Tables XII and XIII, WD++ consistently outperforms other

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

methods in drift detection accuracy. It also achieves competitive detection delay (Table IV and V) and efficient resource usage (Table XVI). Fig. 10 visualizes temporal patterns, with blue bars representing raw data, trend lines indicating behavior, and grey regions marking drift detected by WD++.

A reliable detector is expected to identify historically validated drift events, though additional undocumented drifts may also occur. Table XII evaluates detection within known drift periods, while Table XIII covers the changes in full dataset timelines. Datasets C_{18}–C_{20} show abrupt drift due to short-term disruptions [Fig. 10(a)–(c)], whereas C_{21}–C_{22} exhibit gradual drift over longer durations [Fig. 10(d)–(e)]. Across all datasets, WD++ consistently detected drift events with high accuracy, aligning well with the shaded regions in Fig. 10. Competing methods often missed abrupt drifts in C_{18}–C_{20} and only partially captured gradual drifts in C_{21}–C_{22}. These findings emphasize the robustness and reliability of WD++ in handling both types of drift.

3) MRDs: We evaluate the performance of WD++ on multivariate real-world datasets by examining drift occurrence across individual variables and aggregating the results at the dataset level as presented in Section IV-B3. Specifically, drift detection is first conducted independently for each variable within a multivariate dataset. The outcomes are then integrated using a majority voting scheme to determine whether the dataset as a whole exhibits drift or a subset of it shows drift.

This approach acknowledges the possibility that not all variables in a multivariate stream experience drift simultaneously. By isolating and evaluating the distributional changes in each variable, we aim to ensure a granular yet holistic analysis of drift behavior. The aggregated results computed via variablewise assessments followed by majority voting are reported in Table XIV. Although our method has been designed for drift detection in univariate environment, these findings provide insight into the robustness of our method in detecting multivariate concept drift, especially in scenarios involving heterogeneous data dimensions.

4) MSDs: To further evaluate the performance of WD++ in more complex settings, we conducted experiments using MSDs specifically constructed with embedded, known drift patterns as presented in Section IV-B4. These controlled datasets enable precise assessment of detection accuracy under high-dimensional conditions. As detailed in Table XIV, each variable in the multivariate streams was individually assessed using the WD++ detector. The final drift decision for each time point was then derived using a majority voting scheme, as described in Section IV-C3, ensuring that the aggregated outcome reflects a consensus across all dimensions.

This strategy allows for flexible yet reliable drift detection in multivariate contexts, where changes may affect some variables but not others. The results show that WD++ consistently identifies drift events with high accuracy across all evaluated datasets, demonstrating strong alignment with the ground truth. These findings emphasize the robustness of our method in high-dimensional data environments and confirm its generalizability beyond univariate cases.

5) Timely Detection of Drift: A key measure of a drift detector's effectiveness is its ability to detect drift promptly, identifying changes as they occur and distinguishing between temporary deviations and sustained shifts in the data pattern. While many existing methods delay in identifying abrupt drifts, often requiring prolonged or gradual deviations to trigger detection, WD++ consistently overcomes this limitation. As reflected in the results presented in the last four columns of Table IV, i.e., MTFA, MDR, MTD, and MTTD (for details see Section IV-A), WD++ achieves both rapid and accurate detection across a range of drift scenarios. These findings reinforce its strong performance in scenarios where timeliness is critical, setting it apart from other detectors.

6) Resource Utilization: We evaluated the computational efficiency of WD++ against WD and eight state-of-the-art drift detectors using both real and synthetic datasets as presented in Table VI. We present detailed results of these comparisons in Tables XV–XVIII (Section C-2, supplementary material). These tables include observations across cover average runtime, CPU usage, and memory consumption of each detectors. Methods were ranked by resource usage, with lower usage earning higher ranks. WD was the most efficient, followed closely by WD++, which balanced drift detection performance with low computational overhead. EDDM ranked third in efficiency but showed weaker drift detection accuracy.

The experimental results presented above demonstrate the comparison of the performance of WD++ against baseline methods across a variety of scenarios. We now present a more detailed analysis of our results in the following section.

V. Discussion

The results outlined in the previous section reveal several significant patterns in the performance and behavior of WD++ across different types of concept drift. In this section, we provide a detailed interpretation of these findings, structured around three key aspects: the evaluation of detection accuracy in Section V-A, an analysis of computational efficiency in Section V-B, and an explanation of the superior performance of the majority voting mechanism and the reason of its selection within WD++ in Sections V-C and V-D provides an ablation study on the contribution of each statistical test in drift detection versus the ensemble approach.

A. Detection Accuracy

To evaluate WD++, we compared its detection accuracy against several state-of-the-art drift detectors across six standardized scenarios as shown in Fig. 5. Using paired t-tests and 95% CIs, we assessed statistical significance and the magnitude of performance differences. WD++ consistently outperformed all baseline methods, with statistically significant improvements ($p < 0.05$) in five out of six cases. Fig. 5 ranks methods using p-values, mean differences, and 95% CIs to visualize comparative performance and robustness. Higher ranked methods exhibited consistently superior results across these statistical measures. WD++ delivered the most substantial improvements over $HDDM_A$ (+0.718, $p = 0.005$) and $HDDM_W$ (+0.673, $p =$

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

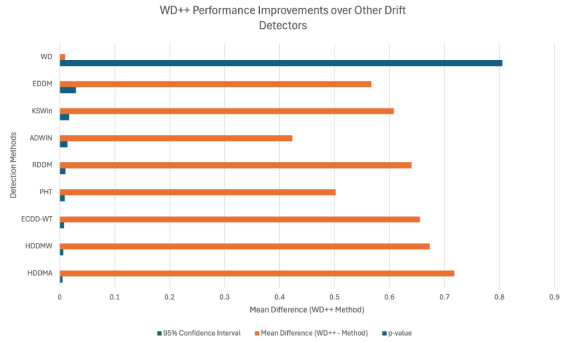


Fig. 5. WD++ performance improvements over other drift detectors (mean differences with 95% CI).

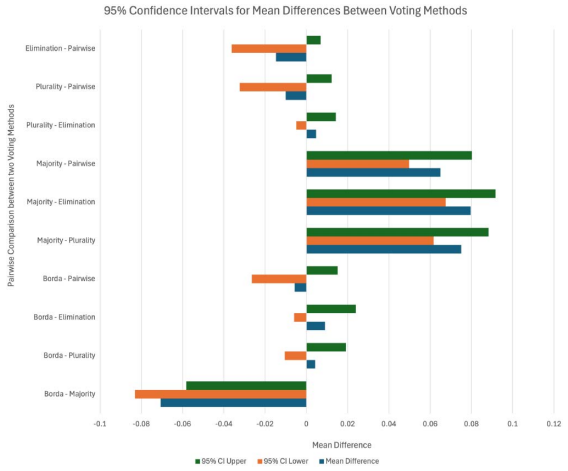


Fig. 6. WD++ performance with different voting methods (mean differences with 95% CI).

0.0065), with CIs entirely above zero, indicating reliable and practically meaningful gains. Comparisons with its predecessor, WD, revealed no significant difference ($p = 0.805$), suggesting comparable performance between the two.

Overall, WD++ proved robust under both abrupt and gradual drift, offering improved accuracy, low false alarms, and efficient detection times, while maintaining low computational overhead. These results support its suitability for real-time and resource-constrained environments, such as smart cities. Furthermore, we observed the improvement in classification accuracy of a Naive Bayes classifier when combined with the drift detection techniques in comparison. We present these results in Table V, demonstrating that WD++ has slightly superior performance to its predecessor WD and other detectors such as ADWin, EDDM, and KSWin.

B. Computational Complexity Analysis

To assess the scalability of WD++ in large-scale data stream environments, we analyzed its complexity based on the key components given in Section III-A, i.e., multilevel sliding windows, ECDF computation, and multiple statistical distance tests. A step-by-step processing of these operations is listed in Table VII. The following discussion focuses on evaluating WD++ in terms of its time and space complexity.

We compare the computational performance of both WD and WD++, revealing that they consistently produce the most competitive results. This is due to the way our techniques leverage ECDF as a sufficient statistic while efficiently discarding irrelevant and stale data. In both modes of WD++, once the ECDF $\hat{F}(x)$ of a distribution is calculated, there is no need to retain the original observations.

A key feature of our technique is winLevSize, which defines which data can be considered irrelevant. For example, when assessing drift in January 2020, there is no need to store data from December 2019. Similarly, when assessing drift between February 2019 and February 2020, January 2019 data become stale and are not needed for further analysis. These characteristics contribute to the efficiency of both WD and WD++, making them lightweight and resource-efficient for drift detection. As a result, these techniques are well suited for integration into any incremental learning algorithm, where computational resources are often limited.

Our analysis indicates that WD continues to outperform WD++ in terms of efficient computational resource usage. This is primarily because WD relies on a single statistical test, i.e., the KS test for drift detection. In contrast, WD++ employs five statistical tests running in parallel, which naturally leads to slightly higher computational resource utilization. While these tests occur concurrently, the increase in resource usage is due to the fact that the additional tests in WD++ involve either multiple data points or areas under the curve between two distributions, as opposed to the single point comparison used in WD with the KS test. Despite this slight increase in resource consumption, WD++ achieves the highest drift detection accuracy compared to all other detectors in Tables VI. This higher computational utilization is a reasonable tradeoff for the significant improvement in drift detection accuracy, especially when compared to the other techniques evaluated.

1) Time Complexity: WD++ performs drift detection by comparing distributions over dynamically indexed sliding windows using five ECDF-based tests (KS, Kuiper, CvM, AD, and EMD). Each detection cycle involves the following.

a) ECDF computation with a complexity of $O(w.logw)$, where w is the size of window on each level, also referred to as $winLevSize$ in Algorithm 1.

b) Five statistical tests, each typically $O(w)$, resulting in a combined complexity of $O(T.w)$, where $T = 5$ is a constant and represents the *number of ECDF-based statistical tests* used in our method.

Thus, the total time complexity per detection cycle is $O(w.logw)$. This cost can be amortized over the data stream if the concept drift detection is performed periodically.

2) Space Complexity: WD++ maintains a fixed number of sliding windows and summary buffers across a small number of hierarchical levels L. Therefore, its space requirements are $O(L.w)$. This is efficient for streaming applications, as both L and w are bounded and do not grow with the total length of the incoming data stream.

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF CLASSIFICATION ACCURACY ON NAIVE BAYES CLASSIFIER ACROSS VARIOUS DRIFT DETECTORS

Data	ADWIN	EDDM	$HDDM_A$	$HDDM_W$	PHT	WD	WD++	KSWin	RDDM	ECDD-WT
C_1–C_5	84.71	85.27	84.98	83.64	83.89	85.02	**86.97**	82.91	79.14	80.07
C_6–C_{10}	**69.81**	64.56	64.99	66.21	64.89	66.92	67.08	63.19	64.01	60.99
C_{11}–C_{17}	66.73	62.97	62.12	68.41	69.67	70.81	**73.02**	64.33	67.11	69.04
C_{18}–C_{22}	60.89	61.24	67.49	62.04	65.28	71.45	**78.98**	63.56	60.09	66.28
C_{23}–C_{26}	61.94	60.13	64.25	65.05	67.11	68.02	**70.12**	62.19	64.08	65.18
C_{27}–C_{30}	61.22	60.43	65.28	66.87	69.03	68.25	**71.24**	71.43	73.05	69.15
Avg. Acc.	67.55	65.76	68.19	68.70	69.98	71.75	**74.57**	67.93	67.91	68.45

Note: Classification accuracy results are given in %. The best individual accuracy is indicated in boldface.

TABLE VI
ESTIMATED UTILISATION OF COMPUTATIONAL RESOURCES

	Real Datasets			Synthetic Datasets		
Methods	Runtime (s)	CPU (%)	RAM (MB)	Runtime (s)	CPU (%)	RAM (MB)
ADWIN[6]	2.821	24.12	163.25	2.925	24.87	164.48
EDDM[4]	2.494	21.35	151.79	2.530	22.08	152.37
$HDDM_A$[5]	2.720	22.61	156.34	2.842	23.21	157.37
$HDDM_W$[6]	2.841	23.98	158.47	2.911	24.55	159.61
PHT[4]	2.531	23.12	152.49	2.690	24.16	153.66
WD[2]	**2.026**	**20.42**	**146.17**	**2.159**	**20.81**	**147.04**
WD++[1]	2.113	21.32	148.27	2.234	21.87	149.87
KSWin[3]	2.113	21.32	148.27	2.234	21.87	149.87
RDDM[4]	2.113	21.32	148.27	2.234	21.87	149.87
ECDD-WT[5]	2.113	21.32	148.27	2.234	21.87	149.87

Note: The best results are given in boldface. Rank for detectors is given next to their names in this table; where the technique with the first rank is the best of all.

TABLE VII
COMPARISON OF THE FIVE STATISTICAL TESTS USED IN WD++ DRIFT DETECTION METHOD

Analysis	Operation	Complexity	Explanation
Time	**Window Update**	$O(1)$	Adding/removing data points to/from sliding windows is constant time.
	Index Computation	$O(1)$	Simple arithmetic mapping of indices for comparisons across levels.
	ECDF Calculation	$O(wlogw)$	For each window, sorting is needed to build ECDFs for statistical tests.
	Statistical Tests	$T.O(w)$	Five ECDF tests are run per detection, each linear in window size (often 5 $\times$ w).
	Voting Decision	$O(T)$	Majority voting over five test outcomes to decide on drift.
Space	**Sliding Windows**	$O(Lw)$	Stores windows at multiple levels.
	Statistical Buffers	$O(w)$	Buffers for ECDF comparisons.
	Flags and Counters	$O(1)$	Minimal metadata.

Note: n: Total number of data records processed; w: Maximum size of a sliding window at any hierarchy level; L: Number of hierarchical window levels (multiscale detection); T: Number of ECDF-based statistical tests (typically T = 5 in WinDrift++).

To summarize, the design of WD++ ensures that both time and space complexity remain scalable and bounded, enabling real-time drift detection in large data streams with limited requirements for computational resources.

C. Voting Methods

As one of the objectives of WD++ is to enhance the robustness of drift detection, it supports multiple voting methods that combine outputs from individual statistical tests. Our default is majority voting that flags drift when over 50% of tests detect change. We evaluated several alternatives including Borda count, plurality voting, elimination, and pairwise comparison. Our experiments (Table VIII, Fig. 6) benchmarked these methods across 30 datasets, measuring performance through standard accuracy metrics and statistical significance tests (e.g., pairwise t-tests as presented in Table IX).

Results show that majority voting offers the best balance between accuracy and computational efficiency, while methods such as Borda and Elimination may offer marginal gains in specific scenarios, such as imbalanced data and running in batch mode. Given its general stability, we adopt majority voting as the default. However, WD++ remains modular, allowing practitioners to adapt or replace the voting mechanism based on application needs. Full results and step-by-step calculations are detailed in the supplementary material (Section B-3).

D. Ablation Study

Next, we evaluate the contribution of individual tests in WD++, we conducted a leave-one-out ablation study by disabling one statistical test at a time from the five-test ensemble as shown in Table X. The objective of our ablation study is to show the performance of WD++ when each statistical test

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
COMPARISON OF VOTING METHODS AND THEIR USE IN WD++

Method	Efficiency	Sensitivity	Complexity	Pros	Cons	Best to Use When
Majority Voting	High	Medium	Low	Simple and fast Low false positive	May miss early drift signals (less sensitive)	Prioritize stability and minimal false alarms
Plurality Voting	Higher	High	Low	More sensitive Can detect drift faster	Higher chance of false positives May overreact to outliers	Early warnings and can tolerate more false alarms
Borda Count	Medium	High	Medium	High confidence scores Balanced detection	Needs richer input (not binary) Medium complexity	Detectors provide more than binary, and detailed decisions are required
Plurality with Elimination	Low	Medium	High	Promotes stronger consensus over multiple rounds	Too slow for real-time, complex implementation	Running in batch mode or need strong consensus
Pairwise Comparison	Low	High	High	Robust to strategic voting Captures detailed relationships	High computational cost, not ideal for rapid streaming decisions	Value fairness and have resources for higher computation

TABLE IX
PAIRWISE COMPARISON OF VOTING METHODS IN WD++ USING MEAN Δ AND 95% CONFIDENCE INTERVALS

Comparison	Mean Δ	95% CI $\downarrow$	95% CI $\uparrow$
Borda—Majority	–0.0707	–0.0831	–0.0582
Borda—Plurality	0.0043	–0.0105	0.0192
Borda—Elimination	0.009	–0.006	0.024
Borda—Pairwise	–0.0057	–0.0264	0.0151
Majority—Plurality	0.075	0.0617	0.0883
Majority—Elimination	0.0797	0.0676	0.0917
Majority—Pairwise	0.065	0.0498	0.0802
Plurality—Elimination	0.0047	–0.0049	0.0143
Plurality—Pairwise	–0.01	–0.0323	0.0123
Elimination—Pairwise	–0.0147	–0.0362	0.0069

TABLE X
COMPARISON OF PERFORMANCE OF WD++ EXCLUDING CERTAIN STATISTICAL TESTS

Config. (4 out of 5 Tests)	Accuracy $\uparrow$	FPR $\downarrow$	Delay $\downarrow$	Time $\downarrow$
WD++ minus KS	0.86	0.09	40	25
WD++ minus Kuiper	0.88	0.08	37	25
WD++ minus CvM	0.89	0.07	35	**24**
WD++ minus AD	0.87	0.08	38	**24**
WD++ minus Wasserstein	0.88	0.07	36	**24**
Full WD++ (All five Tests)	**0.92**	**0.06**	**32**	29

Note: The best individual result is indicated in boldface.

is individually removed and compared the performance of an ensemble versus individual detection. Results indicate that the full ensemble of five tests consistently yields the best performance across all metrics (higher accurately detected drift, lower FPR, lower mean delay between actual drift and detection, and average run time in milliseconds).

Excluding KS or AD from the ensemble slightly degrades accuracy and increases detection delay, underscoring their critical role in maintaining robustness. This highlights the value of ensemble diversity in WD++'s performance. Results are averaged across all synthetic and real-world datasets.

To ensure fair evaluation in ablation scenarios with only four active tests, we apply a conservative tie-breaking rule: in the event of a 2–2 vote split, the system defaults to "no drift." This choice helps reduce FPs and is consistently used across all configurations. While WD++ allows this behavior to be user-configurable, we recommend using an odd number of tests in practice to avoid ambiguity, as previously explained in Section III-A3.

VI. CONCLUSION AND FUTURE DIRECTIONS

In this article, we presented WD++, a novel and efficient concept drift detection framework for nonstationary time series data. WD++ integrates hierarchical sliding windows at multiple temporal scales with a diverse ensemble of ECDF-based statistical distance tests (e.g., KS, CvM, AD, Kuiper, and EMD), enabling robust detection of both abrupt and gradual drifts. Our experimental results on a wide suite of synthetic and real-world datasets demonstrate that WD++ achieves superior detection accuracy and lower FPRs compared to eight state-of-the-art detectors. Furthermore, our evaluation of ensemble decision strategies shows that majority voting provides a favorable trade-off between performance and computational simplicity.

While WD++ adopts fixed hierarchical window sizes to maintain consistency and low overhead, this structure may limit adaptability in environments with highly dynamic drift patterns. As a direction for future work, we plan to investigate ADWIN mechanisms that adjust window size based on the observed drift characteristics. We also aim to extend WD++ to handle noisy or irregular data and enable finer grained classification of drift types.

DATA AVAILABILITY STATEMENT

Code and datasets are available on https://github.com/naureenaqvi/windriftplusplus.

REFERENCES

[1] N. Naqvi, S. U. Rehman, and M. Z. Islam, "A hyperconnected smart city framework: Digital resources using enhanced pedagogical techniques," *Australas. J. Inf. Syst.*, vol. 24, pp. 1–42, 2020.
[2] G. Ditzler and R. Polikar, "Incremental learning of concept drift from streaming imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 10, pp. 2283–2301, Oct. 2013.
[3] N. Naqvi, S. U. Rehman, and M. Z. Islam, "WinDrift: Early detection of concept drift using corresponding and hierarchical time windows," in *Proc. Australas. Conf. Data Mining*, Singapore: Springer, 2022, pp. 73–89.
[4] J. A. Gama, I. Žliobait, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surveys (CSUR)*, vol. 46, no. 4, pp. 1–37, 2014.
[5] N. Lu, G. Zhang, and J. Lu, "Concept drift detection via competence models," *Artif. Intell.*, vol. 209, pp. 11–28, 2014.
[6] J. Shao, Z. Ahmadi, and S. Kramer, "Prototype-based learning on concept-drifting data streams," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2014, pp. 412–421.
[7] U. Ali and T. Mahmood, "A novel framework for concept drift detection using autoencoders for classification problems in data streams," *Int. J. Mach. Learn. Cybern.*, vol. 16, no. 1, pp. 397–418, 2025.

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.

[8] P. Lu, J. Lu, A. Liu, and G. Zhang, "Early concept drift detection via prediction uncertainty," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 18, 2025, pp. 19124–19132.

[9] I. Frias-Blanco, J. del Campo-Ávila, G. Ramos-Jimenez, R. Morales-Bueno, A. Ortiz-Diaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 3, pp. 810–823, Mar. 2015.

[10] A. Liu, "Concept drift adaptation for learning with streaming data," Ph.D. dissertation, Univ. Technol. Sydney, Australia, 2018.

[11] V. M. Souza, A. R. Parmezan, F. A. Chowdhury, and A. Mueen, "Efficient unsupervised drift detector for fast and high-dimensional data streams," *Knowl. Inf. Syst.*, vol. 63, no. 6, pp. 1497–1527, 2021.

[12] J. Li, K. Malialis, and M. M. Polycarpou, "Autoencoder-based anomaly detection in streaming data with incremental learning and concept drift adaptation," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Piscataway, NJ, USA: IEEE, 2023, pp. 1–8.

[13] H. Raza, G. Prasad, and Y. Li, "EWMA model based shift-detection methods for detecting covariate shifts in non-stationary environments," *Pattern Recognit.*, vol. 48, no. 3, pp. 659–669, 2015.

[14] H. Wang and Z. Abraham, "Concept drift detection for streaming data," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Ireland. Piscataway, NJ, USA: IEEE, 2015, pp. 1–9.

[15] C. Alippi, G. Boracchi, and M. Roveri, "Hierarchical change-detection tests," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 2, pp. 246–258, Feb. 2017.

[16] S. Yu and Z. Abraham, "Concept drift detection with hierarchical hypothesis testing," in *Proc. SIAM Int. Conf. Data Mining*, Philadelphia, PA, USA: SIAM, 2017, pp. 768–776.

[17] Z. Chen, M. Han, H. Wu, M. Li, and X. Zhang, "A multi-level weighted concept drift detection method," *J. Supercomput.*, vol. 79, no. 5, pp. 5154–5180, 2023.

[18] M. Han, D. Mu, A. Li, S. Liu, and Z. Gao, "Concept drift detection methods based on different weighting strategies," *Int. J. Mach. Learn. Cybern.*, vol. 15, no. 10, pp. 4709–4732, 2024.

[19] H. Mehmood, P. Kostakos, M. Cortes, T. Anagnostopoulos, S. Pirttikangas, and E. Gilman, "Concept drift adaptation techniques in distributed environment for real-world data streams," *Smart Cities*, vol. 4, no. 1, pp. 349–371, 2021.

[20] H. Guo, H. Li, Q. Ren, and W. Wang, "Concept drift type identification based on multi-sliding windows," *Inf. Sci.*, vol. 585, pp. 1–23, 2022.

[21] W. J. Faithfull, J. J. Rodríguez, and L. I. Kuncheva, "Combining univariate approaches for ensemble change detection in multivariate data," *Inf. Fusion*, vol. 45, pp. 202–214, 2019.

[22] M. A. Stephens, "Introduction to Kolmogorov on the empirical determination of a distribution," in *Breakthroughs in Statistics.* New York, NY, USA: Springer, 1992, pp. 93–105.

[23] M. N. Adnan and M. Z. Islam, "Forest CERN: A new decision forest building technique," in *Proc. Adv. Knowl. Discovery Data Mining: 20th Pacific-Asia Conf., PAKDD*, Auckland, New Zealand, April 19–22, 2016, Part I 20. New Zealand: Springer, 2016, pp. 304–315.

[24] M. N. Adnan and M. Z. Islam, "Forest PA: Constructing a decision forest by penalizing attributes used in previous trees," *Expert Syst. Appl.*, vol. 89, pp. 389–403, 2017.

[25] N. H. Kuiper, "Tests concerning random points on a circle," *Nederl. Akad. Wetensch. Proc. Ser. A*, vol. 63, pp. 38–47, 1960.

[26] D. A. Darling, "The Kolmogorov-Smirnov, Cramer-Von Mises tests," *Ann. Math. Statist.*, vol. 28, no. 4, pp. 823–838, 1957.

[27] P. A. Lewis, "Distribution of the Anderson-Darling statistic," *Ann. Math. Statist.*, vol. 32, no. 4, pp. 1118–1124, Dec. 1961.

[28] L. N. Vaserstein, "Markov processes over denumerable products of spaces, describing large systems of automata," *Problemy Peredachi Informatsii*, vol. 5, no. 3, pp. 64–72, 1969.

[29] A. W. Van der Vaart, *Asymptotic Statistics*, 3rd ed. Cambridge, U.K.: Cambridge Univ. Press, 2000.

[30] K. Aslansefat, S. Kabir, A. Abdullatif, V. Vasudevan, and Y. Papadopoulos, "Toward improving confidence in autonomous vehicle software: A study on traffic sign recognition systems," *Computer*, vol. 54, no. 8, pp. 66–76, 2021.

[31] D. S. Wilks, *Statistical Methods in the Atmospheric Sciences*, 100th ed. USA: Academic Press, 2011.

[32] G. Van Rossum and F. L. Drake, Jr., *Python Reference Manual.* Amsterdam, The Netherlands: Centrum Voor Wiskunde en Informatica, 1995.

[33] M. Baena-Garcıa, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, and R. Morales-Bueno, "Early drift detection method," in *Proc. 4th Int. Workshop Knowl. Discovery Data Streams*, vol. 6, 2006, pp. 77–86.

[34] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proc. 7th SIAM Int. Conf. Data Mining. SIAM*, Apr. 2007, pp. 135–150.

[35] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[36] C. Raab, M. Heusinger, and F.-M. Schleif, "Reactive soft prototype computing for concept drift streams," *Neurocomputing*, vol. 416, pp. 340–351, 2020.

[37] R. S. Barros, D. R. Cabral, P. M. Gonçalves, Jr., and S. G. Santos, "RDDM: Reactive drift detection method," *Expert Syst. Appl.*, vol. 90, pp. 344–355, 2017.

[38] G. J. Ross, N. M. Adams, D. K. Tasoulis, and D. J. Hand, "Exponentially weighted moving average charts for detecting concept drift," *Pattern Recognit. Lett.*, vol. 33, no. 2, pp. 191–198, 2012.

[39] J. Montiel, J. Read, A. Bifet, and T. Abdessalem, "Scikit-multiflow: A multi-output streaming framework," *J. Mach. Learn. Res.*, vol. 19, no. 1, pp. 2915–2914, 2018.

[40] J. C. Sisniega and Á. L. García, "Frouros: A Python library for drift detection in machine learning problems," 2022, *arXiv:2208.06868*.

[41] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over f1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, pp. 1–13, 2020.

[42] A. Bifet, J. Read, B. Pfahringer, G. Holmes, and I. Žliobaitė, "CD-MOA: Change detection framework for massive online analysis," in *Adv. Int. Data Anal. XII – Proc. Int. Symp. Int. Data Anal. (IDA 2013)*, Lecture Notes in Computer Science, vol. 8207, Berlin, Heidelberg, Germany: Springer, 2013, pp. 92–103

[43] A. Pesaranghader, H. L. Viktor, and E. Paquet, "McDiarmid drift detection methods for evolving data streams," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Piscataway, NJ, USA: IEEE, 2018, pp. 1–9.

[44] T. Nishio, "The impact of sports events on inbound tourism in New Zealand," *Asia Pacific J. Tourism Res.*, vol. 18, no. 8, pp. 934–946, 2013.

[45] A. J. Martin, A. N. Eagleman, and P. Pancoska, "A case study of regional sport organization development in triathlon," *J. Appl. Sport Manage.*, vol. 6, no. 1, p. 8, 2014.

[46] D. Johnston et al., "The 2010/2011 Canterbury earthquakes: Context and cause of injury," *Natural Hazards*, vol. 73, no. 2, pp. 627–637, 2014.

[47] U.S. Bureau of Labor Statistics, "Spotlight on Statistics: The Recession of 2007–2009 – Unemployment Demographics," Feb. 2012.

[48] P. M. Downes, K. Hanslow, and P. Tulip, "The effect of the mining boom on the Australian economy," Reserve Bank Aust. Res. Discussion Paper, no. 2014-08, 2014.

[49] J. A. Blackard and D. J. Dean, "Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables," *Comput. Electron. Agric.*, vol. 24, no. 3, pp. 131–151, 1999.

[50] M. Harries and N. S. Wales, "Splice-2 comparative evaluation: Electricity pricing," *Technical Report UNSW-CSE-TR9905, School of Computer Science & Engineering*, University of New South Wales, Sydney, Australia, July 1999.

[51] V. Souza, D. M. d Reis, A. G. Maletzke, and G. E. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," *Data Mining Knowl. Discovery*, 2020.

[52] W. N. Street and Y. Kim, "A streaming ensemble algorithm (sea) for large-scale classification," in *Proc. 7th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2001, pp. 377–382.

[53] R. Agrawal, T. Imielinski, and A. Swami, "Database mining: A performance perspective," *IEEE Trans. Knowl. Data Eng.*, vol. 5, no. 6, pp. 914–925, Dec. 1993.

[54] H. M. Gomes et al., "Adaptive random forests for data stream classification," *Mach. Learn.*, vol. 106, pp. 1469–1495, Jun. 2017.

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.



Naureen Naqvi (Member, IEEE) received the B.E. degree in electronic engineering from NED University of Engineering and Technology, Karachi, Pakistan, the bachelor's (Hons.) degree in computing from Charles Sturt University, Bathurst, Australia, in 2020, and the M.E. degree in computer systems engineering from NED University of Engineering and Technology, Karachi, Pakistan, in 2013. She is currently working toward the Ph.D. degree in knowledge discovery and data mining from Charles Sturt University, Bathurst, Australia, in 2026.

She has more than 18 years of experience in product development, innovation, digital transformation, and portfolio management across consulting, enterprise, and start-up environments. She has cofounded and advised several start-ups, led large-scale technology transformations, and secured multiple innovation patents in artificial intelligence-enabled product and data-driven solution design. Her research interests include data science, machine learning, artificial intelligence, cloud computing, and privacy and security, with a focus on stream analytics, classification, and adaptive learner models under concept drift. She has also explored the application of emerging technologies such as the Internet of Things, smart cities, renewable energy, and cloud operations to support innovation and digital transformation. She actively mentors start-ups, particularly in the social sector, and provides corporate training in project management, data science, and data storytelling. She is passionate about fostering innovation through customer-centric, resource-efficient, and scalable technology solutions.



Sabih Ur Rehman (Member, IEEE) received the Bachelor of Engineering (Hons.) degree in electronics and telecommunication engineering from the University of South Australia, Adelaide, Australia, in 1998, and the Ph.D. degree in the area of wireless sensor networks from Charles Sturt University, Australia.

He is currently a Senior Lecturer and the Course Director of the School of Computing, Mathematics and Engineering, Charles Sturt University. He has extensive industry experience in delivering large-scale networking solutions along with providing system administration, security, and data integration services. His research expertise is in the areas of wireless channel modeling, network planning, routing and switching, and big data analytics. His research interests include the domains of the Internet of Things (IoT), knowledge discovery through data mining, and distributed network security via blockchain-enabled solutions, especially in the spheres of intelligent transport systems, environmental sustainability, e-health, and precision agriculture; with the aim to positively influence social, economic, and environmental sustainability of communities in rural Australia via digitally enabled solutions.

Dr. Rehman regularly publishes his research and serves as a reviewer for several respected journals and conferences. He is a member of the Engineers Australia (MIEAust) and the Australian Computer Society (ACS).



Md Zahidul Islam (**Zahid Islam**) is a Professor in computer science with the Faculty of Business, Justice and Behavioural Sciences (FOBJBS), Charles Sturt University, Australia. His research interests include data mining, privacy, cybersecurity, and applications of data mining in real life.

Prof. Islam currently serves as an Associate Dean of Research in FOBJBS. For more information, see https://sites.google.com/view/prof-zahid-islam.

Authorized licensed use limited to: Universidad de Antioquia. Downloaded on October 12,2026 at 17:21:18 UTC from IEEE Xplore. Restrictions apply.